

Learning and Reconstructing Conflicts in O-RAN: A Graph Neural Network Approach

Arshia Zolghadr, Joao F. Santos, Luiz A. DaSilva, Jacek Kibilda

Commonwealth Cyber Initiative, Virginia Tech, USA, e-mail: {arshiaz,joaosantos,ldasilva,jkibilda}@vt.edu

Abstract—The Open Radio Access Network (O-RAN) architecture enables the deployment of third-party applications on the RAN Intelligent Controllers (RICs). However, the operation of third-party applications in the Near Real-Time RIC (Near-RT RIC), known as xApps, may result in conflicting interactions. Each xApp can independently modify the same control parameters to achieve distinct outcomes, which has the potential to cause performance degradation and network instability. The current conflict detection and mitigation solutions in the literature assume that all conflicts are known a priori, which does not always hold due to complex and often hidden relationships between control parameters and Key Performance Indicators (KPIs). In this paper, we introduce the first data-driven method for reconstructing and labeling conflict graphs in O-RAN. Specifically, we leverage GraphSAGE, an inductive learning framework, to dynamically learn the hidden relationships between xApps, parameters, and KPIs. Our numerical results, based on a conflict model used in the O-RAN conflict management literature, demonstrate that our proposed method can effectively reconstruct conflict graphs and identify the conflicts defined by the O-RAN Alliance.

Index Terms—O-RAN, Conflict Detection, Near-RT RIC, xApps, Graph Neural Networks

I. INTRODUCTION

The Open Radio Access Network (O-RAN) Alliance introduced an open architecture that disaggregates the RAN into different functional components orchestrated under a management framework capable of running custom third-party applications [1]. This management framework comprises two RAN Intelligent Controllers (RICs) operating in different timescales. The Non Real-time RIC (Non-RT RIC) is a component of the Service Management and Orchestration (SMO) and is in charge of hosting third-party applications, known as rApps, that implement long-term tasks, with timescales greater than 1000 ms, e.g., data analytics, AI/ML model training, and inference to optimize the RAN [2]. In contrast, the Near Real-Time RIC (Near-RT RIC) is in charge of hosting third-party applications, known as xApps, that implement time-sensitive tasks, with timescales on the order of 10–1000 ms.

In the Near-RT RIC, the xApps act as plugin-like extensions that can control base stations to enhance the capabilities of the RAN and provide network operators with different functionality, e.g., load balancing, mobile handover, and RAN slicing [3]. The xApps may operate independently from each other, which has the potential for conflicting interactions, as different plugins may modify the same control parameters to achieve distinct outcomes. For example, a plugin may optimize parameters to maximize throughput, whereas another plugin aims to minimize energy consumption, leading to performance degradation and

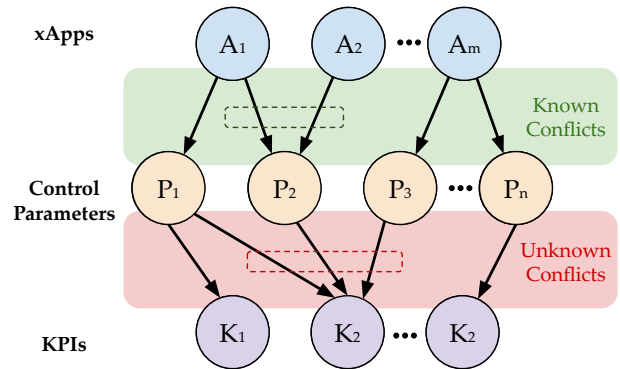


Fig. 1: Conflict graph illustrating potential known conflicts between xApps and their control parameters (identifiable before deployment), and unknown conflicts that may arise due to the non-trivial relationships between control parameters and KPIs. reduced network stability and reliability [4].

The O-RAN specifications identify three types of conflict that can occur within the Near-RT RIC [1]: direct, implicit, and indirect. These conflicts can then be defined in terms of their level of interaction required and observability. *Direct conflicts* arise when plugins simultaneously attempt to modify the same control parameter, which can be identified before their deployment. For instance, one xApp may aim to optimize throughput by increasing transmit power, while another xApp may attempt to minimize energy consumption by reducing it. *Indirect conflicts* arise when xApps modify distinct parameters that inadvertently affect the same Key Performance Indicator (KPI), which are less obvious and cannot always be anticipated. For example, different xApps can modify the Cell Individual Offset (CIO) and antenna tilts, control parameters that affect a single KPI: the cell's handover boundary. Finally, *implicit conflicts* emerge when a parameter implicitly influences another by affecting KPIs used by plugins to trigger further actions on other control parameters, leading to complex relationships that are difficult to observe. For example, a plugin modifying the RAN slice allocation to affect user throughput may unintentionally interfere with another plugin monitoring the user throughput to control handovers.

Despite growing industry and regulatory interest in O-RAN deployments and the recent surge in related research efforts, few works [4]–[7] explore conflict detection and mitigation. For instance, both [5] and [6] focused on fundamental approaches to conflict mitigation, assuming that all relationships between xApps, control parameters, and KPIs are known a priori. The

relationship between xApps and parameters is known by design, since xApps explicitly inform the Subscription Manager on the Near-RT RIC of their subscriptions to modify control parameters [3]. However, the relationship between control parameters and KPIs is often non-trivial, dynamic, and scenario-dependent [7], and such, indirect and implicit conflicts may be unknown in advance, as shown in Fig. 1.

To tackle this challenge, [4] and [7] proposed conflict detection solutions where conflicts are not necessarily known in advance. The work of [7] proposed an xApp prioritization strategy that manually groups control parameters affecting the same KPIs, and identifies conflicts within the group when a predefined performance degradation is observed. However, this approach is only suitable for the detection of direct and indirect conflicts. PACIFISTA [4] proposed modeling the relationships between xApps, control parameters, and KPIs as heterogeneous conflict graphs. This approach enables the detection of conflicts by analyzing the structure of the graph to identify well-defined topologies representing different types of conflicts. To create conflict graphs, they adopt a profiling pipeline that tests and makes a statistical profile of xApps in a sandbox environment, similar to a digital twin. However, the effectiveness of their conflict graph (and consequently conflict detection) relies on the quality of an arbitrary set of tests and the fidelity of the sandbox to capture a realistic environment.

In this work, we build upon the conflict graph modeling approach introduced in [4], addressing its reliance on arbitrary testing within digital twins. We propose a novel data-driven method using Graph Neural Networks (GNNs) to learn the relationships between the nodes (e.g., xApps, control parameters, and KPIs) based on collected data from the current state of the xApps, parameters, and KPIs. By leveraging GNNs, we can predict links and identify hidden relationships based on the graph topology and node features, allowing us to reconstruct complete conflict graphs. Specifically, we leverage GraphSAGE [8], an inductive learning framework, for its ability to generalize to unseen nodes and scale efficiently across large graphs. In possession of the reconstructed conflict graph, we can utilize graph labeling to identify the different types of conflicts based on their well-defined, graph-based definition. This approach allows us to detect conflicts that emerge from the learned interactions between control parameters and KPIs.

The main contributions of this paper are as follows:

- We propose a data-driven, GNN-based method for reconstructing conflict graphs, capturing sequential interactions between the nodes of the conflict graph, e.g. where xApps modify parameters, parameters affect KPIs, and KPIs provide feedback to xApps.
- We propose graph-based definitions of conflicts, based on the definitions proposed in [4], for identifying the three types of conflicts considered by the O-RAN Alliance.
- We validate our conflict graph reconstruction and evaluate our conflict labeling solutions using Gaussian distributed KPIs generated from a conflict model used in [5], [9].

To the best of our knowledge, this is the first work to propose

a conflict graph reconstruction method for learning unknown relationships and conflicts in O-RAN. The remainder of this paper is organized as follows. In Section II, we pose our problem statement on the reconstruction of conflict graphs. In Section III, we detail our GNN-based method to learn from complex relationships from collected data and reconstruct conflict graphs. In Section IV, we describe graph-based definitions for the three types of conflicts in O-RAN. In Section V, we validate our solution numerically using a conflict model from the literature. Finally, in Section VI, we summarize our findings and discuss avenues for future work.

II. PROBLEM STATEMENT

We define a heterogeneous graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ where $\mathcal{V}$ is a set of vertices and $\mathcal{E}$ is a set of edges. The topological structure of $\mathcal{G}$ can be represented by an adjacency matrix $\mathbf{A}$. We assume that $\mathcal{G}$ is a vertex-labeled graph and that there are three categories of vertices: (i) plugins deployed in the Near-RT RIC (denoted as $\mathcal{A}$); (ii) control parameters that can be modified by plugins (denoted as $\mathcal{P}$); and (iii) RAN KPIs that can be affected by changes to parameters (denoted as $\mathcal{K}$). Therefore, $\mathcal{V} = \mathcal{A} \cup \mathcal{P} \cup \mathcal{K}$. In addition, an edge between $\mathcal{A}$, $\mathcal{P}$, and $\mathcal{K}$ can be interpreted as a valid relationship between the vertices. For instance, the existence of an edge between $\mathcal{A}$ and $\mathcal{P}$ or $\mathcal{A}$ and $\mathcal{K}$ represents the subscription of xApps to control parameters or monitor KPIs, respectively. Similarly, an edge between $\mathcal{P}$ and $\mathcal{K}$ shows the influence of a parameter on a KPI. While some relationships in the heterogeneous graph representation of conflicts are known by design, i.e., the relationships between $\mathcal{A}$ and $\mathcal{P}$ and between $\mathcal{A}$ and $\mathcal{K}$ through the subscription process between xApps and the Near-RT RIC's Subscription Manager [1] [3], others are less straightforward. Specifically, the relationships between $\mathcal{P}$ and $\mathcal{K}$ are often non-trivial, dynamic, and scenario-dependent, and must be inferred from the collected data from the current state of the xApps, parameters, and KPIs [4]. Therefore, our objective is to estimate the adjacency matrix $\mathbf{A}$ based on the collected data.

III. CONFLICT GRAPH RECONSTRUCTION IN O-RAN

We propose a GNN-based approach for reconstructing conflict graphs based on collected data. To accomplish this, we leverage GraphSAGE [8], an inductive learning framework that learns vertex embeddings by aggregating and transforming feature information from neighboring vertices. Unlike conventional neural networks that process inputs independently, GraphSAGE requires a graph structure as input, making it particularly suitable for capturing complex relationships that evolve over time as the system progresses. To represent the data from the RAN, we construct a multivariate time series temporal graph $\mathcal{G}_T = (\mathcal{V}_T, \mathcal{E}_T)$, where each vertex $v_t \in \mathcal{V}_T$ represents time step t and is associated with a feature vector $\mathbf{x}_t$, containing concatenated values of $\mathcal{P}$ and $\mathcal{K}$ observed at time step t . We assume a temporal dependency limited to consecutive vertices, i.e., that our features at time step t only affect the values of features at $t + 1$, and as such, we define $(v_t, v_{t+1}) \in \mathcal{E}_T$. This representation structure is inspired by temporal graph tasks, where time-stamped data is modeled as a graph to capture

relationships across time steps [10].

The temporal graph serves as the structural backbone for the GraphSAGE neural network, where each vertex v_t is mapped to a node, and its feature vector $\mathbf{x}_t$ becomes the input attribute for the first layer of the GNN. Since we assume a temporal dependency limited to consecutive vertices, the neighborhood $N(v_t)$ of vertex v_t only includes the previous time step v_{t-1} and the next time step v_{t+1} . The GraphSAGE framework processes the graph interactively in layers, where each layer updates the feature embedding of each vertex v_t by aggregating information from its neighbors. At each layer k , the feature embedding of vertex v_t denoted as $\mathbf{h}_{v_t}^{(k)}$, is computed as following the approach in [8]:

$$\mathbf{h}_{v_t}^{(k)} = \sigma \left(\mathbf{W}^k \cdot \mathbb{E} \left[\{\mathbf{h}_{v_t}^{(k-1)}\} \cup \{\mathbf{h}_u^{(k-1)} : \forall u \in N(v_t)\} \right] \right), \quad (1)$$

where $\mathbf{W}^k \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ is the learnable weight matrix at layer k with input dimension d_{in} and output dimension d_{out} , σ is the non-linear activation function, $\mathbb{E}$ represents the element-wise mean of the neighborhood $N(v_t)$, and the $(\cdot)$ symbol represents the matrix multiplication operation. Consequently, the embeddings at the final layer, $\mathbf{h}_v^{(K)}$, where K denotes the number of layers, represent the learned latent representations of the vertices. We then train the model using the Mean Squared Error (MSE) loss to minimize the difference between the reconstructed and input embeddings. The loss function is defined as:

$$L_{\text{MSE}} = \frac{1}{|\mathcal{V}_T|} \sum_{t=1}^T \left\| \mathbf{h}_{v_t}^{(K)} - \mathbf{x}_t \right\|^2, \quad (2)$$

where T is the set of all timestamps, $\mathbf{x}_t$ is the input feature vector at time t , $|\mathcal{V}_T|$ is the number of vertices, $\|\cdot\|^2$ denotes the squared euclidean norm, and $\mathbf{h}_{v_t}^{(K)}$ is the predicted embedding of vertex v_t .

Once our training is complete, the final embedding from the last training epoch encodes both the temporal relationships and feature dependencies. These embeddings are used to compute pairwise correlation across all features that describe the desired conflict graph. Finally, we apply a fixed threshold to binarize the correlations and obtain a reconstructed adjacency matrix $\hat{\mathbf{A}}$, representing the conflict graph [11]. The fixed threshold is empirically chosen to balance sensitivity to weak correlations in the reconstructed conflict graph. In post-processing, we complete our reconstructed adjacency matrix by leveraging the known information about the subscription of xApps to parameters and KPIs, i.e., populating the matrix with existing edges between $\mathcal{A}$ and $\mathcal{P}$ as well as $\mathcal{A}$ and $\mathcal{K}$.

IV. LABELING CONFLICTS IN O-RAN

Based on the heterogeneous graph $\mathcal{G}$, we introduce definitions for the three types of conflicts considered by the O-RAN Alliance [12]. These extend the definitions proposed in [4] by diving deeper into the chains of complex relationships that give rise to indirect conflicts between different parameters, allowing us to understand their logical dependencies, as shown in Fig. 2. This approach captures subtle and nuanced dependencies that

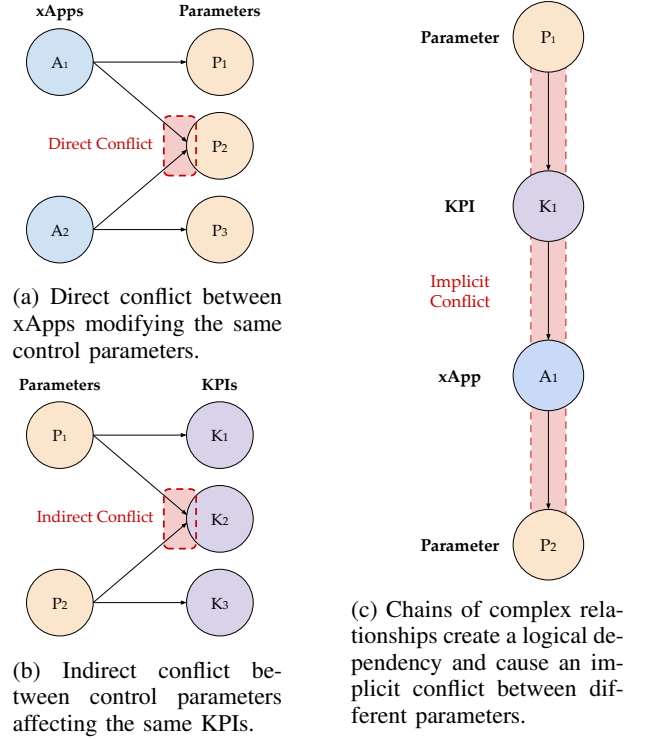
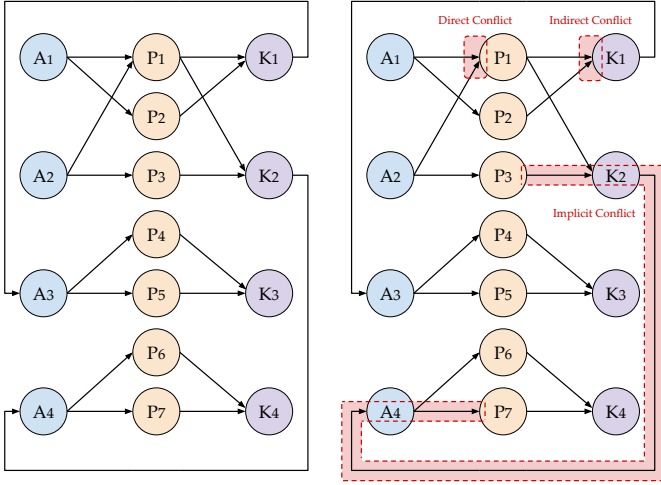


Fig. 2: Illustration of the types of conflicts considered by the O-RAN Alliance using our graph labeling definitions, allowing us to identify conflicts in the structure of a conflict graph $\mathcal{G}$. might otherwise be overlooked, while also enabling the potential identification of the root causes of the indirect conflict.

- **Direct Conflicts** For any $a_i, a_j \in \mathcal{A}$, where $i \neq j$, a_i and a_j are in direct conflict if $e(a_i, p), e(a_j, p) \in \mathcal{E}$. An example of two applications subscribing and attempting to modify the same parameter is shown in Fig. 2a.
- **Indirect Conflicts** For any $a_i, a_j \in \mathcal{A}$ controlling $p_m, p_n \in \mathcal{P}$ respectively, and a KPI $k \in \mathcal{K}$, a_i and a_j are in indirect conflict if $e(a_i, p_m), e(a_j, p_n), e(p_m, k), e(p_n, k) \in \mathcal{E}$. An example of two parameters affecting the same KPI k_2 is shown in Fig. 2b.
- **Implicit Conflicts** Considering any $a_i, a_j \in \mathcal{A}$ modifying $p_m, p_n \in \mathcal{P}$, respectively, and a KPI $k \in \mathcal{K}$, a_i and a_j are in implicit conflict if $e(a_i, p_m), e(p_m, k), e(k, a_j), e(a_j, p_n) \in \mathcal{E}$. This type of conflict cannot be observed directly, and even the relationship between the xApps and parameters may not be obvious, as shown in Fig. 2c.

Remarks:

- The definition of an implicit conflict implies a depth of conflicts, whereby the existence of an implicit conflict between a_i and a_j , and a_j and a_k implies that there is an implicit conflict also between a_i and a_k . In this preliminary work, we limit ourselves to identifying the depth-one implicit conflicts only.
- The existence of an implicit conflict between a_i and a_j (or p_m and p_n) implies a dependency between a_i and a_j (or p_m and p_n), which can be interpreted as an indirect conflict as assumed in [4]. Notwithstanding the merits of



(a) Exemplar conflict graph we adopt in our numerical evaluation. (b) Example of conflict labeling according to our definitions.

Fig. 3: Structure of the conflict model [9] we used to validate our solutions for graph reconstruction and conflict labeling.

alternative conflict labeling approaches, we adhere to the conflict labels defined by the O-RAN specifications [12].

V. NUMERICAL EVALUATION

In this section, we validate our GNN-based method for reconstructing conflict graphs and detecting conflicts. First, we describe the conflict model and dataset used to train our GNN, and the metric used in our evaluations. Then, we assess the accuracy of our GNN to reconstruct conflict graphs and our graph labeling solution to detect different types of conflicts.

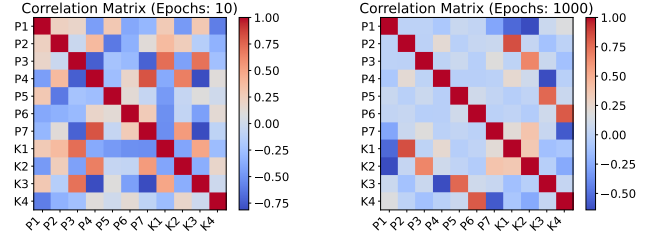
A. Dataset, Network Architecture, and Evaluation Metric

To evaluate our proposed GNN-based method for learning the relationships between xApps, control metrics, and KPIs, and reconstructing conflict graphs, we collected a dataset based on the conflict model for cognitive autonomous networks proposed in [9] and later adopted in other works on conflict management in O-RAN [5]. Fig. 3 shows a graphical representation of the conflict model and an example of the conflict labeling based on the definitions from Section IV. The conflict model shows the interaction between four xApps that control seven parameters, which, in turn, influence four KPIs. We assume the KPIs follow a Gaussian distribution, with values defined by equations that describe the relationships between control parameters and KPIs, as follows: $K_1 = 0.5 \cdot \exp\left(-\frac{(P_1+50)^2}{(2P_2)^2}\right)$, $K_2 = \exp\left(-\frac{(P_1-50)^2}{(2P_3)^2}\right)$, $K_3 = \exp\left(-\frac{(P_4+K_1)^2}{(2P_5)^2}\right)$, $K_4 = \exp\left(-\frac{(P_7+K_2)^2}{(2P_6)^2}\right)$. Based on this model, we sampled control parameters using uniformly distributed random values within the ranges specified in Table I and calculated their effects on the corresponding KPI to generate datasets (of arbitrary size) that captured their correlations.

To capture the unknown relationships in the conflict model, we designed a GNN with 11 features, representing its seven parameters and four KPIs. The neural network architecture consists of three layers, where the input and output layers have sizes equal to the size of the dataset, and the hidden layer has

Parameter	P_1	P_2	P_3	P_4	P_5	P_6	P_7
Value Range	[0, 300]	[0, 300]	[0, 3]	[0, 3]	[0, 3]	[0, 3]	[0, 3]

TABLE I: Ranges of values used for parameters in our dataset.



(a) Early stages of the correlation matrix at 10 epochs, showing the results of limited learning time showing the model's convergence to learn the correct relationships between parameters and KPIs. (b) Correlation matrix with longer training time at 1000 epochs, results of limited learning time showing the model's convergence to learn the correct relationships between parameters and KPIs.

Fig. 4: Evolution of the correlation matrix of our GNN model with the number of epochs (created with a dataset size of 450 samples), illustrating the convergence.

16 neurons. We also used a learning rate of 0.001.

To evaluate our reconstruction and labeling, we adopt the F1 Score, the harmonic mean of precision and recall, as shown in Eq. 5. This metric provides a balanced measure of accuracy due to its capacity to capture both false positives and negatives.

$$\text{Precision} = \frac{\text{True Positive}}{\text{True Positive} + \text{False Positive}}, \quad (3)$$

$$\text{Recall} = \frac{\text{True Positive}}{\text{True Positive} + \text{False Negative}}, \quad (4)$$

$$\text{F1 Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \quad (5)$$

B. Graph Reconstruction Accuracy

In analysis, we are interested in assessing our GNN's ability to learn the relationships and accurately reconstruct conflict graphs. The performance of a GNN model depends both on the amount of training information, i.e., the dataset size, and the training time, as the number of epochs allows the GNN model to iteratively update its weights and better capture the underlying patterns from the graph. Fig. 4 presents the output correlation matrix generated by our GNN model for different numbers of epochs, showing the benefits of additional training. However, some spurious edges with low correlation values persist even with additional epochs. While further training could reduce these edges, there is a risk of overfitting the GNN to the dataset. To address this, we introduce a cutoff threshold for the correlation values, excluding edges below this threshold from the reconstructed conflict graph.

In Fig. 5, we evaluate our GNN model reconstruction accuracy using the F1 Score for different dataset sizes and a number of epochs, with a fixed threshold of 0.5. To establish a baseline for comparison, we evaluate our GNN-based method for graph reconstruction against a uniformly distributed random graph. We can observe the training time significantly influences the accuracy of the reconstructed graph up to 400 epochs, while the dataset size displays a bigger contribution to the accuracy of

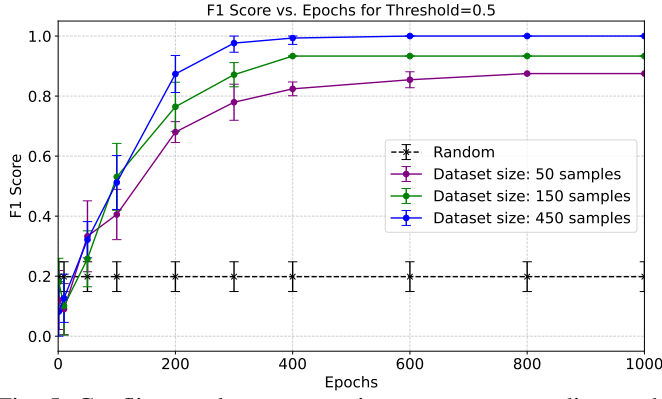


Fig. 5: Conflict graph reconstruction accuracy according to the number of epochs and dataset size for a fixed threshold of 0.5.

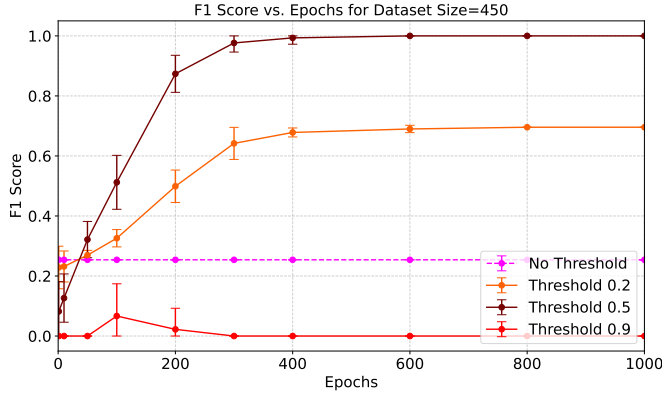


Fig. 6: Conflict graph reconstruction accuracy according to the epochs and thresholds for a fixed dataset size of 450 samples.

the model beyond this point. It is worth mentioning that we observed no substantial accuracy improvement with larger dataset sizes. These results show that our GNN model can successfully learn the relationships and recreate conflict graphs, achieving an accuracy of 100% with at least 450 samples and 600 epochs. In Fig. 6, we evaluate the reconstruction accuracy of our GNN for different thresholds and numbers of epochs, with a fixed dataset size of 450 samples. We observe that training time significantly impacts the accuracy of the reconstructed graph up to 400 epochs, while the threshold becomes more significant beyond this point. These results demonstrate that incorporating a threshold improves accuracy by eliminating false positives caused by edges with low correlation values, achieving an accuracy of 100% with a threshold of 0.5 and 600 epochs.

C. Graph Labeling Accuracy

In this analysis, we are interested in assessing the accuracy of our conflict labeling to identify different types of conflicts on a reconstructed conflict graph. The performance of the conflict labeling depends on the quality of the reconstructed conflict graph to capture the correct structure of the conflict model. Thus, we evaluate the conflict labeling based on the reconstructed graphs from the previous section. We only display results evaluating the identification of implicit and indirect conflicts, as we consistently obtained 100% accuracy for direct conflicts across all settings, and the direct conflicts can be trivially identified through the analysis of the subscription

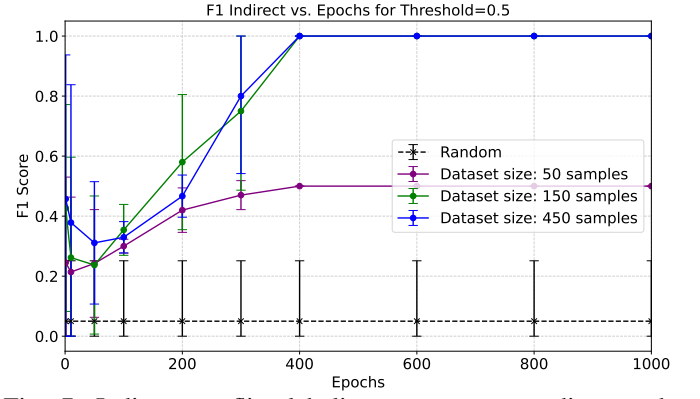


Fig. 7: Indirect conflict labeling accuracy according to the number of epochs and dataset size for a fixed threshold of 0.5.

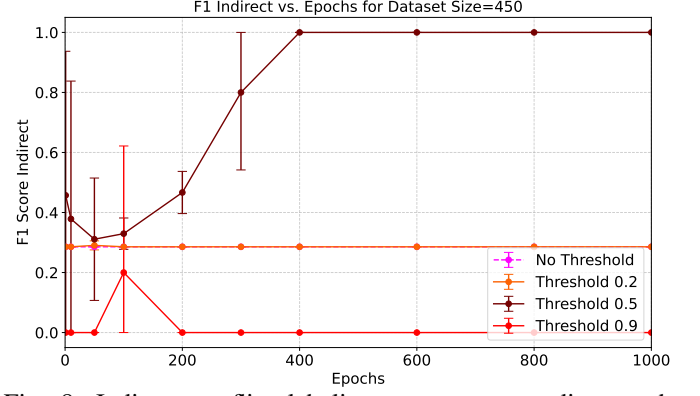


Fig. 8: Indirect conflict labeling accuracy according to the epochs and thresholds for a fixed dataset size of 450 samples.

information on the Near-RT RIC's Subscription Manager [3].

In Fig. 7, we evaluate our graph labeling to detect indirect conflicts on reconstructed conflict graphs created using different dataset sizes and a number of epochs, with a fixed threshold of 0.5. We observe that detecting indirect conflicts is considerably more challenging due to the effect of spurious edges on the models' accuracy. This complexity requires conflict graphs with higher reconstruction accuracy to capture correct edges without spurious edges giving false positive accuracy. Thus, accurately identifying indirect conflicts requires reconstructing conflict graphs with significantly longer training times, at least 600 epochs. In Fig. 8, we evaluate our graph labeling to detect indirect conflicts on reconstructed conflict graphs created using different thresholds and numbers of epochs, with a fixed dataset size of 450 samples. We observe that the threshold plays a crucial role in improving the accuracy of implicit conflict detection. Due to the non-linear relationships between parameters and KPIs in our example model, many variables exhibit low correlations, leading to numerous spurious edges that result in false positives. Our results show that a relatively high threshold of 0.5 offers a good balance to filter out edges with low correlation values to prune false positives and keep meaningful edges to prevent false negatives.

In Fig. 9, we evaluate our graph labeling to detect implicit conflicts on reconstructed conflict graphs created using different dataset sizes and number of epochs, with a fixed threshold of

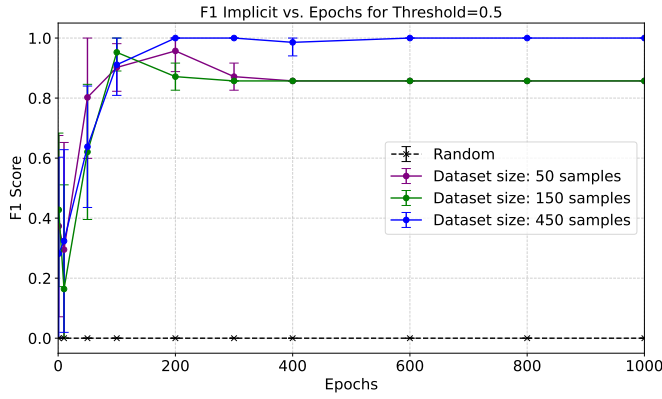


Fig. 9: Implicit conflict labeling accuracy according to the number of epochs and dataset size for a fixed threshold of 0.5.

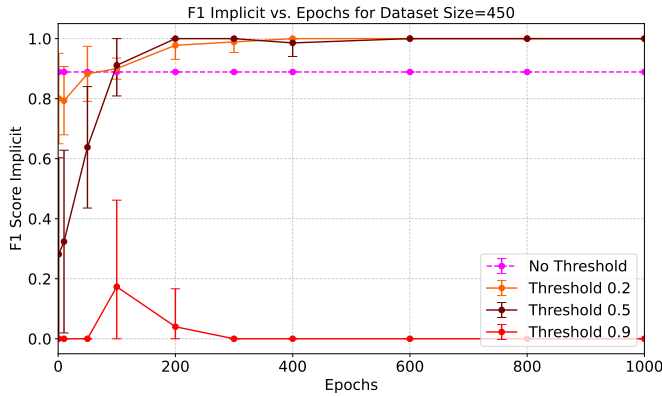


Fig. 10: Implicit conflict labeling accuracy according to epochs and threshold values for a fixed dataset size of 450 samples.

0.5. As expected, the labeling performance improves with the accuracy of the reconstructed graph until 200 epochs (88% reconstruction accuracy), and from there onward, conflict graphs generated with more data points display better performance. In Fig. 10, we evaluate our graph labeling to detect implicit conflicts on reconstructed conflict graphs created using different thresholds and numbers of epochs, with a fixed dataset size of 450 samples. While converging akin to the previous plot, i.e., stabilizing after 300 epochs, we note that the threshold has only a minor impact on the labeling accuracy of implicit conflicts, provided it is not set very high, which can result in false negatives. Our results show that we can achieve a 100% detection of implicit conflicts when the graph is reconstructed with at least 450 samples, 200 epochs, and a threshold of 0.5.

VI. CONCLUSION

In this paper, we proposed the first data-driven GNN-based method for reconstructing conflict graphs in O-RAN. Our approach is designed to learn the relationships between xApps, control parameters, and KPIs to reconstruct complete conflict graphs that capture the three types of conflicts considered by the O-RAN Alliance. In addition, we introduce a conflict labeling solution based on graph-based definitions for direct, indirect, and implicit conflicts. We validated our conflict graph reconstruction and evaluated our conflict labeling using a dataset generated from a known conflict model adopted in other works for conflict management in O-RAN, and demonstrated

its accuracy across different dataset sizes, thresholds, and training epochs. Our results highlighted our ability to achieve high accuracy in graph reconstruction and conflict detection, particularly for implicit and indirect conflicts. It is worth mentioning that data-driven approaches such as ours may yield false negatives in cases where an xApp subscribes to a KPI but does not actively use it to modify parameters, which might be a limitation in certain operational scenarios. In future works, we plan to extend our GNN formulation to reconstruct causal relationships between xApps, parameters, and KPIs, as a more effective approach to mitigating spurious edges and decreasing our dependency on correlation. In addition, we plan to explore the scalability of our solution for different numbers of xApps, parameters, and KPIs, leveraging empirical data obtained in an experimental setting. Finally, we aim to complement our work by developing conflict mitigation techniques to create a complete conflict management system for O-RAN.

ACKNOWLEDGMENTS

The research leading to this paper received support from the Commonwealth Cyber Initiative, an investment in the advancement of cyber R&D, innovation, and workforce development. For more information, visit: www.cyberinitiative.org. This work also received support from the Horizon Europe SNS JU program under grant No. 101139194 (6G-XCEL), and from the National Science Foundation US-Ireland R&D Partnership program under grant No. 2421362.

REFERENCES

- [1] M. Polese *et al.*, "Understanding O-RAN: Architecture, Interfaces, Algorithms, Security, and Research Challenges," *IEEE Communications Surveys & Tutorials (COMST)*, vol. 25, no. 2, pp. 1376–1411, 2023.
- [2] S. D'Oro *et al.*, "OrchestRAN: Network Automation through Orchestrated Intelligence in the Open RAN," in *IEEE Conference on Computer Communications (INFOCOM)*, 2022, pp. 270–279.
- [3] J. F. Santos *et al.*, "Managing O-RAN Networks: xApp Development from Zero to Hero," 2024. [Online]. Available: <https://arxiv.org/abs/2407.09619>
- [4] P. B. del Prever *et al.*, "PACIFISTA: Conflict Evaluation and Management in Open RAN," 5 2024. [Online]. Available: <http://arxiv.org/abs/2405.04395>
- [5] A. Wadud *et al.*, "Conflict Management in the Near-RT-RIC of Open RAN: A Game Theoretic Approach," in *IEEE International Conferences on Internet of Things (iThings) and IEEE Green Computing & Communications (GreenCom) and IEEE Cyber, Physical & Social Computing (CPSCom) and IEEE Smart Data (SmartData) and IEEE Congress on Cybermatics (Cybermatics)*, 2023, pp. 479–486.
- [6] H. Zhang *et al.*, "Team Learning-Based Resource Allocation for Open Radio Access Network (O-RAN)," in *IEEE International Conference on Communications (ICC)*, 2022, pp. 4938–4943.
- [7] C. Adamczyk and A. Kliks, "Conflict Mitigation Framework and Conflict Detection in O-RAN Near-RT RIC," *IEEE Communications Magazine (ComMag)*, vol. 61, no. 12, pp. 199–205, 2023.
- [8] W. Hamilton *et al.*, "Inductive Representation Learning on Large Graphs," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [9] A. Banerjee *et al.*, "Toward Control and Coordination in Cognitive Autonomous Networks," *IEEE Transactions on Network and Service Management (TNSM)*, vol. 19, no. 1, pp. 49–60, 2021.
- [10] N. Xu *et al.*, "TimeGNN: Temporal Dynamic Graph Learning for Time Series Forecasting," in *Springer International Conference on Complex Networks and Their Applications (COMPLEX NETWORKS)*, 2023, pp. 87–99.
- [11] S. Blyth, "Karl Pearson and the Correlation Curve," *JSTOR International Statistical Review (ISR)*, pp. 393–403, 1994.
- [12] O-RAN Working Group 3, "Near-RT RIC Architecture," O-RAN Alliance, Tech. Rep., Jun. 2024.